1- **CLASS TASKS SOLUTIONS**

--- TASK 1, 2, 3 & 4 SOLUTION KEY (PYTHON CODE) ---

```python
import pandas as pd

# Task 1 & 2 Setup
orders_table = {
    "order_id": ["ORD-501", "ORD-502", "ORD-503", "ORD-504", "ORD-505"],
    "client_id": ["CLI-10", "CLI-20", "CLI-30", "CLI-40", "CLI-50"],
    "spend_usd": [320.00, 75.00, 890.00, 210.00, 540.00]
}
df_orders = pd.DataFrame(orders_table)

promo_table = {
    "client_id": ["CLI-10", "CLI-30", "CLI-50", "CLI-60"],
    "promo_code": ["PROMO-A", "PROMO-B", "PROMO-A", "PROMO-C"]
}
df_promo = pd.DataFrame(promo_table)

# Task 2: Inner vs Left Join
inner_merged = pd.merge(df_orders, df_promo, on="client_id", how="inner")
print("--- INNER JOIN ---")
print(inner_merged) # 3 rows (CLI-10, CLI-30, CLI-50)

left_merged = pd.merge(df_orders, df_promo, on="client_id", how="left")
print("\n--- LEFT JOIN ---")
print(left_merged) # 5 rows, with NaNs for CLI-20 and CLI-40 promo_code

# Task 3: Fill NaNs & Groupby Aggregation
left_merged["promo_code"] = left_merged["promo_code"].fillna("No Promo")

promo_summary = left_merged.groupby("promo_code").agg(
    total_spend=("spend_usd", "sum"),
    mean_spend=("spend_usd", "mean"),
    order_count=("order_id", "count")
)

print("\n--- PROMO SUMMARY ---")
print(promo_summary)
```

--- TASK 4 BUSINESS MEMO SOLUTION KEY ---

"To: Liam Vance, Marketing Manager Re: Campaign Promotional Performance Audit

By performing a Left Join across transaction and promo logs, we determined that `PROMO-A` generated the highest total revenue ($860.00 across 2 orders). Customers using `PROMO-A` averaged $430.00 per order, significantly outperforming non-promotional orders ($142.50 average spend). Recommendation: Scale `PROMO-A` distribution while auditing the effectiveness of `PROMO-B`."

2- **HOMEWORK SOLUTION**

```python
# Homework Problem 1 & 2 Python Solution
import pandas as pd

shipment_logs = {
    "shipment_id": ["SHP-11", "SHP-12", "SHP-13", "SHP-14", "SHP-15", "SHP-16", "SHP-17", "SHP-18"],
    "vendor_id": ["VEND-A", "VEND-B", "VEND-A", "VEND-C", "VEND-B", "VEND-C", "VEND-A", "VEND-B"],
    "transit_days": [3.5, 7.2, 4.0, 2.1, 8.5, 2.5, 3.8, 6.9],
    "freight_cost": [400.0, 1200.0, 450.0, 210.0, 1350.0, 230.0, 420.0, 1100.0]
}
df_shipments = pd.DataFrame(shipment_logs)

warehouse_assignments = {
    "shipment_id": ["SHP-11", "SHP-12", "SHP-13", "SHP-14", "SHP-15", "SHP-16", "SHP-17"],
    "destination_warehouse": ["North-DC", "South-DC", "North-DC", "East-DC", "South-DC", "East-DC", "North-DC"],
    "manager_on_duty": ["Sarah", "David", "Sarah", "Michael", "David", "Michael", "Sarah"]
}
df_warehouses = pd.DataFrame(warehouse_assignments)

# Problem 1: Left Join
df_full_log = pd.merge(df_shipments, df_warehouses, on="shipment_id", how="left")
print("Merged Shape:", df_full_log.shape) # (8, 6)
print("Missing Warehouse Count:\n", df_full_log.isnull().sum())

# Problem 2: Impute & Groupby
df_full_log["destination_warehouse"] = df_full_log["destination_warehouse"].fillna("Unassigned")
df_full_log["manager_on_duty"] = df_full_log["manager_on_duty"].fillna("Unassigned")

vendor_summary = df_full_log.groupby("vendor_id").agg(
    avg_transit_days=("transit_days", "mean"),
    total_freight=("freight_cost", "sum")
)

warehouse_summary = df_full_log.groupby("destination_warehouse").agg(
    avg_transit_days=("transit_days", "mean")
)

print("\n--- VENDOR SUMMARY ---")
print(vendor_summary)
print("\n--- WAREHOUSE SUMMARY ---")
print(warehouse_summary)
```

- **Homework Grading Criteria:**
    - Correctly executes a Left Join, retaining all 8 shipments and flagging `SHP-18` as unassigned (`NaN`).

- Identifies **VEND-B** as the high-latency, high-cost vendor (averaging 7.53 days and highest freight spend).
- Identifies **South-DC** as the slowest destination warehouse (averaging 7.85 transit days).
- Formulates strategic recommendations to review logistics contracts with VEND-B and streamline routing to South-DC.